Python Programs 35 – 45

Lab Record: Problem Statement to Conclusion

Name	**V.Ramakumari**
Roll Number	**26000068**
Section	**CSE 1**

Contents

35. Text Compression Using Character Runs
36. Repeated Word and Phrase Detection
37. Nested Loop Calendar Generator
38. ATM Menu-Driven Banking Simulation
39. Multi-Stage Examination Eligibility System
40. Traffic Flow and Congestion Analysis
41. Nested List Classroom Occupancy Analysis
42. Student Marks Matrix Analysis
43. Multi-Branch Sales Matrix Comparison
44. Hospital Ward Occupancy Monitoring
45. Weekly Electricity Consumption Matrix

35. Text Compression Using Character Runs

1. Problem Statement

Encode a string by grouping consecutive repeated characters and compare original and encoded lengths, including cases where encoding is not beneficial.

2. Objective

Build a run-length encoded string (character + count) and decide whether it is shorter than the original.

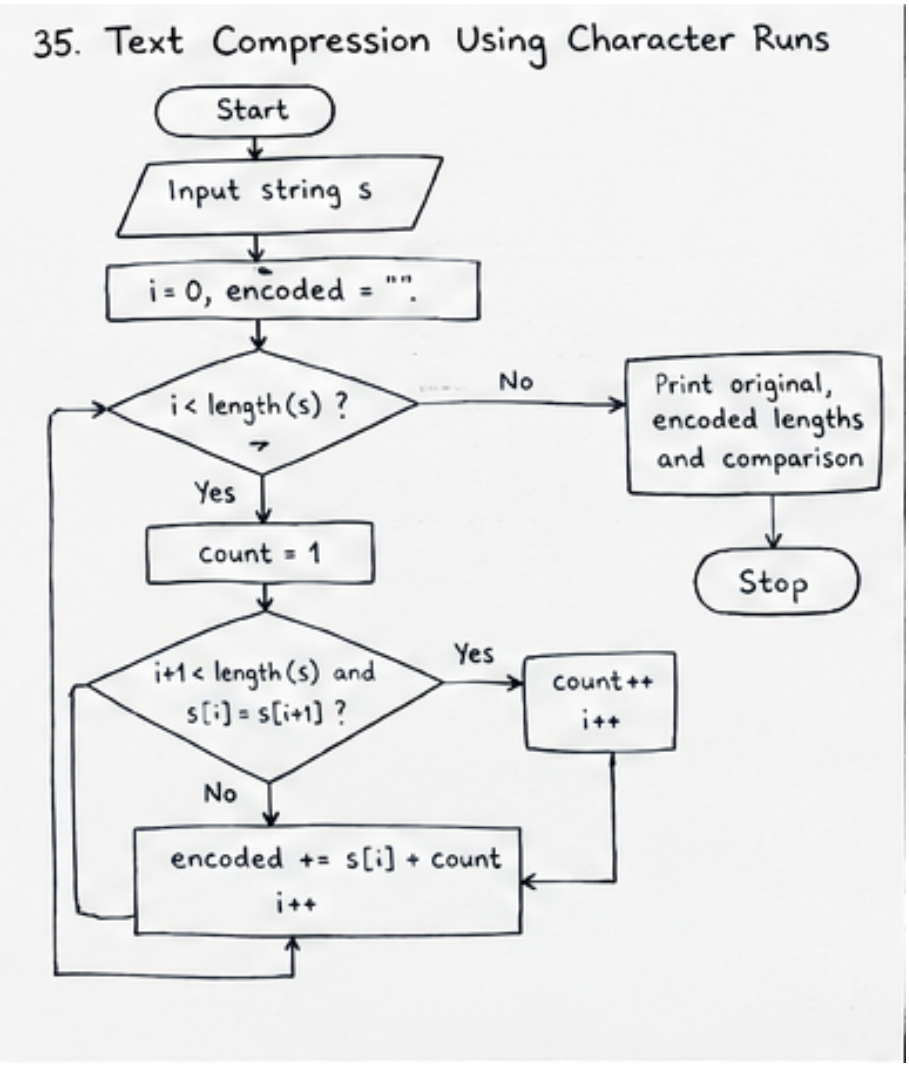
3. Assumptions

Input is a non-empty string; counts are written as digits after each character; encoding is beneficial only if encoded length < original length.

4. Algorithm

1. Read text
2. Set encoded = empty, i = 0
3. While i < length: count = 1; while next char equals current, count++ and i++
4. Append char + count to encoded; i++
5. Print both strings and lengths
6. If encoded length >= original length print 'not beneficial', else 'beneficial'

5. Flowchart



6. Pseudocode

```
READ text
encoded <- "" ; i <- 0
WHILE i < LEN(text)
  count <- 1
  WHILE i+1 < LEN(text) AND text[i] = text[i+1]
    count++ ; i++
  encoded <- encoded + text[i] + STR(count)
  i++
PRINT text, encoded, lengths
IF LEN(encoded) >= LEN(text) PRINT "not beneficial"
ELSE PRINT "beneficial"
```

7. Pvthon Source Code

```
text = input("Enter a string: ")

encoded = ""
i = 0

while i < len(text):
    count = 1

    while i + 1 < len(text) and text[i] == text[i + 1]:
        count += 1
        i += 1

    encoded += text[i] + str(count)
    i += 1

print("Original string:", text)
print("Encoded string:", encoded)
print("Original length:", len(text))
print("Encoded length:", len(encoded))

if len(encoded) >= len(text):
    print("Encoding is not beneficial")
else:
    print("Encoding is beneficial")
```

8. Test Cases

#	Input	Expected result
1	ramakumari	r1a1m1a1k1u1m1a1r1i1, 10 → 20, not beneficial
2	aaabbbcc	a3b3c2, 8 → 6, beneficial
3	aaaaaaaaaa	a10, 10 → 3, beneficial
4	abc	a1b1c1, 3 → 6, not beneficial

9. Output

Code 35.py

Code

35.py

```
1   text = input("Enter a string: ")
2
3   encoded = ""
4   i = 0
5
6   while i < len(text):
7       count = 1
8
9       while i + 1 < len(text) and text[i] == text[i + 1]:
10          count += 1
11          i += 1
12
13      encoded += text[i] + str(count)
14      i += 1
15
16  print("Original string:", text)
17  print("Encoded string:", encoded)
18  print("Original length:", len(text))
19  print("Encoded length:", len(encoded))
20
21  if len(encoded) >= len(text):
22      print("Encoding is not beneficial")
23  else:
24      print("Encoding is beneficial")
```

Output

Problems Output Debug Console Terminal Ports

```
PS C:\Users\HP\OneDrive\Desktop\CSE-1\Dr.RVR NRI D
ive/Desktop/CSE-1/Dr.RVR NRI DTBU(Geethu)/35.py"
Enter a string: ramakumari
Original string: ramakumari
Encoded string: r1a1m1a1k1u1m1a1r1i1
Original length: 10
Encoded length: 20
Encoding is not beneficial
```

10. Conclusion

The program correctly run-length encodes any string and, by comparing lengths, shows that compression helps only when characters repeat in long runs.

36. Repeated Word and Phrase Detection

1. Problem Statement

Process a paragraph, normalize letter case, count word occurrences using permitted string and list operations, and identify repeated words and their positions.

2. Objective

Find every word that occurs more than once in a paragraph and report all its positions (1-based), ignoring case and punctuation.

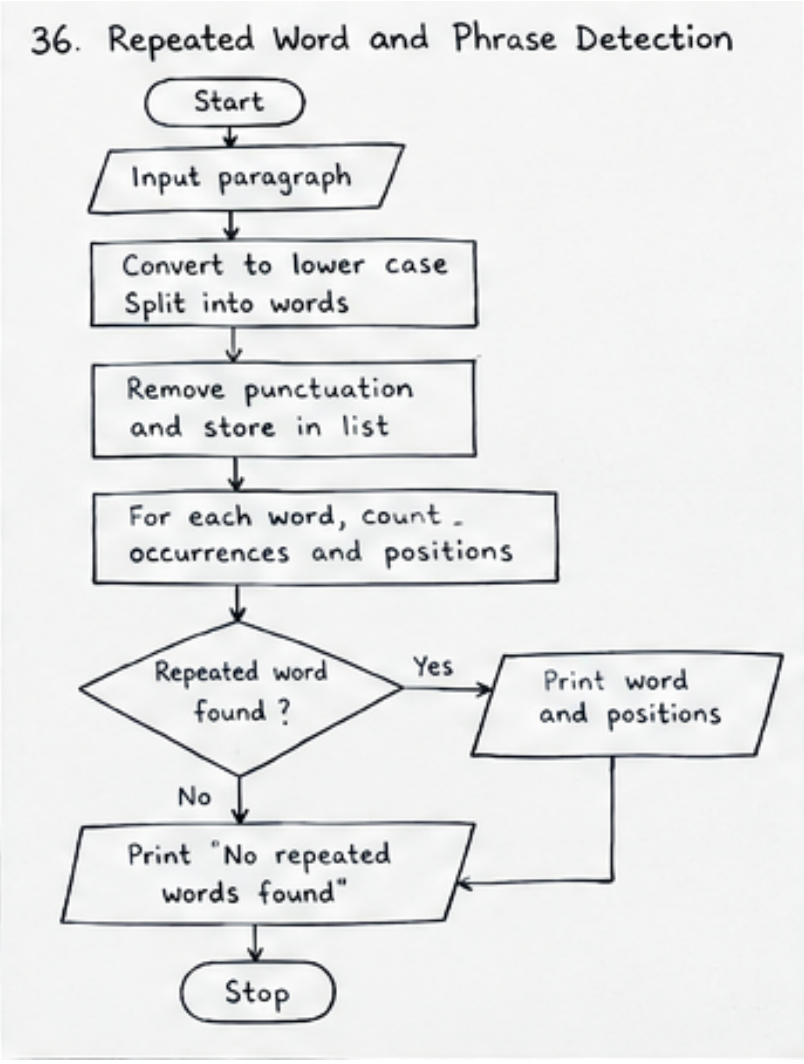
3. Assumptions

Words are separated by spaces; punctuation . , ! ? ; : at word ends is ignored; positions start at 1.

4. Algorithm

1. Read paragraph, convert to lower case, split into words
2. Strip punctuation from each word into clean_words
3. For each word, skip it if already seen earlier
4. Collect all positions where it occurs
5. If more than one position, print word and positions
6. If none found print 'No repeated words found'

5. Flowchart



6. Pseudocode

```
READ text ; words <- SPLIT(LOWER(text))
FOR each word: strip punctuation, add to clean_words
found <- False
FOR i in 0..n-1
  IF clean_words[i] seen before: CONTINUE
  positions <- all j+1 where clean_words[j] = word
  IF LEN(positions) > 1: PRINT word, positions ; found <- True
IF NOT found PRINT "No repeated words found"
```

7. Pvthon Source Code

```
text = input("Enter a paragraph: ")
words = text.lower().split()

clean_words = []

for word in words:
    word = word.strip(".,!?;:")
    clean_words.append(word)

print("Repeated words and positions:")

found = False

for i in range(len(clean_words)):
    word = clean_words[i]

    if word in clean_words[:i]:
        continue

    positions = []

    for j in range(len(clean_words)):
        if clean_words[j] == word:
            positions.append(j + 1)

    if len(positions) > 1:
        print(word, "appears at positions", positions)
        found = True

if not found:
    print("No repeated words found")
```

8. Test Cases

#	Input	Expected result
1	Python is easy and Python is useful	python [1, 5]; is [2, 6]
2	Hello world	No repeated words found
3	The cat. the dog, THE end	the appears at positions [1, 3, 5] (case/punctuation ignored)

9. Output

Code 36.py

Code

```python
text = input("Enter a paragraph: ")
words = text.lower().split()

clean_words = []

for word in words:
    word = word.strip(".,!?;:")
    clean_words.append(word)

print("Repeated words and positions:")

found = False

for i in range(len(clean_words)):
    word = clean_words[i]

    if word in clean_words[:i]:
        continue

    positions = []

    for j in range(len(clean_words)):
        if clean_words[j] == word:
            positions.append(j + 1)

    if len(positions) > 1:
        print(word, "appears at positions", positions)
        found = True

if not found:
    print("No repeated words found")
```

Output

```
PS C:\Users\HP\OneDrive\Desktop\CSE-1\Dr.RVR NRI DTBU(
ive/Desktop/CSE-1/Dr.RVR NRI DTBU(Geethu)/36.py"
Enter a paragraph: Python is easy and Python is useful
Repeated words and positions:
python appears at positions [1, 5]
is appears at positions [2, 6]
```

10. Conclusion

The program detects repeated words independent of case and punctuation and reports every position, or a clear message when there are none.

37. Nested Loop Calendar Generator

1. Problem Statement

Generate a formatted monthly calendar using the supplied starting weekday and number of days, correctly aligning weeks and handling valid month lengths.

2. Objective

Print a Monday-to-Sunday calendar grid with dates aligned under the correct weekday using nested loops.

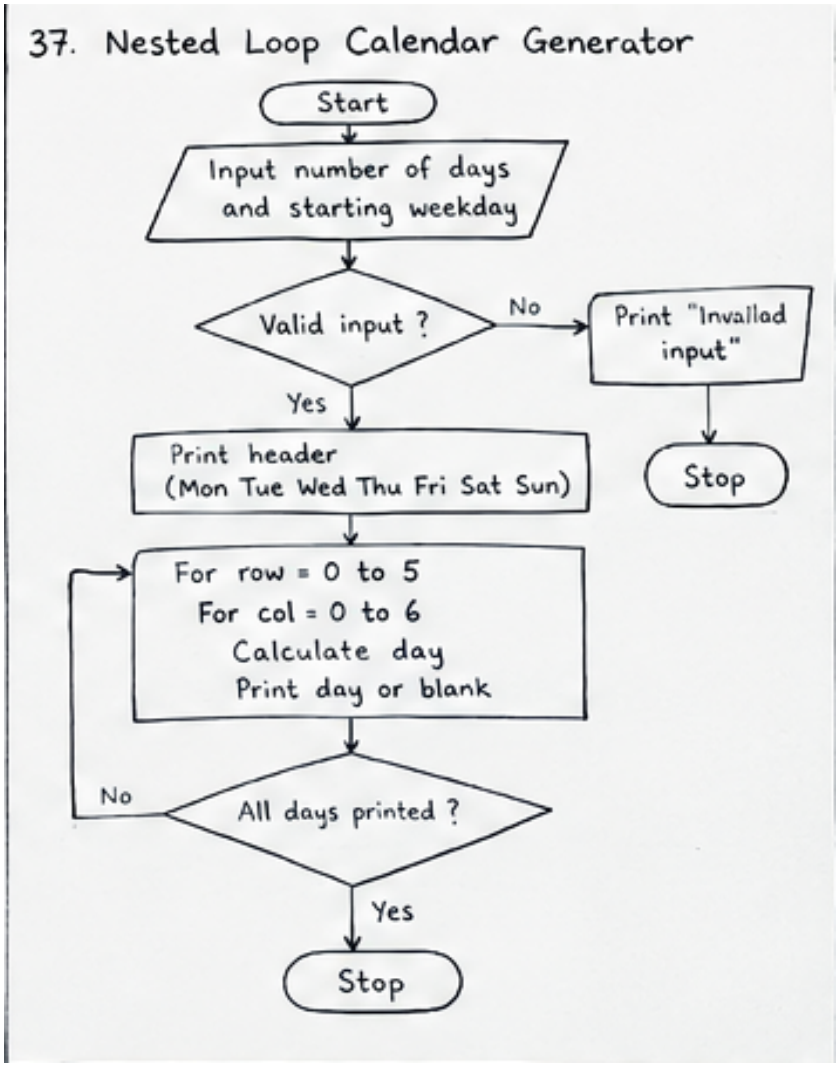
3. Assumptions

Days must be 28–31; starting weekday is 0–6 (0 = Mon … 6 = Sun); a month fits in at most 6 week rows.

4. Algorithm

1. Read days and start weekday
2. If days outside 28–31 or start outside 0–6, print 'Invalid input'
3. Print header Mon…Sun
4. For each row (0–5) and column (0–6) compute day = row*7 + col – start + 1
5. Print day (width 3) if 1 ≤ day ≤ days, else blanks
6. Stop after the row that contains the last day

5. Flowchart



6. Pseudocode

```
READ days, start
IF days<28 OR days>31 OR start<0 OR start>6: PRINT "Invalid input"
ELSE
  PRINT header
  FOR row 0..5
    line <- ""
    FOR col 0..6
      day <- row*7 + col - start + 1
      IF 1<=day<=days line += day ELSE line += blanks
    PRINT line
    IF row*7+7-start >= days BREAK
```

7. Pvthon Source Code

```
days = int(input("Enter number of days in month: "))
start = int(input("Enter starting weekday (0-6): "))

if days < 28 or days > 31 or start < 0 or start > 6:
    print("Invalid input")
else:
    print("Mon Tue Wed Thu Fri Sat Sun")

    for row in range(6):
        line = ""

        for col in range(7):
            day = row * 7 + col - start + 1

            if day >= 1 and day <= days:
                line += f"{day:3} "
            else:
                line += "    "

        print(line)

        if row * 7 + 7 - start >= days:
            break
```

8. Test Cases

#	Input	Expected result
1	30 days, start 2	1 under Wed … 30 under Thu (as in output)
2	28 days, start 0	1 under Mon; exactly 4 full rows
3	31 days, start 6	1 under Sun; 6 rows; last row 30 31
4	32 days, start 2	Invalid input
5	30 days, start 7	Invalid input

9. Output

Code 37.py

Code

```python
days = int(input("Enter number of days in month: "))
start = int(input("Enter starting weekday (0-6): "))

if days < 28 or days > 31 or start < 0 or start > 6:
    print("Invalid input")
else:
    print("Mon Tue Wed Thu Fri Sat Sun")

    for row in range(6):
        line = ""

        for col in range(7):
            day = row * 7 + col - start + 1

            if day >= 1 and day <= days:
                line += f"{day:3} "
            else:
                line += "    "

        print(line)

        if row * 7 + 7 - start >= days:
            break
```

Output

```
PS C:\Users\HP\OneDrive\Desktop\CSE-1\Dr.RVR NRI DTB
ive/Desktop/CSE-1/Dr.RVR NRI DTBU(Geethu)/37.py"
Enter number of days in month: 30
Enter starting weekday (0-6): 2
Mon Tue Wed Thu Fri Sat Sun
          1   2   3   4   5
  6   7   8   9  10  11  12
 13  14  15  16  17  18  19
 20  21  22  23  24  25  26
 27  28  29  30
```

10. Conclusion

The calendar is correctly aligned for every valid weekday/month length, and invalid inputs are rejected.

38. ATM Menu-Driven Banking Simulation

1. Problem Statement

Implement repeated menu operations for deposits, withdrawals, balance inquiries, and transaction summaries, with validation and specified account limits.

2. Objective

Simulate an ATM that loops until Exit, validating amounts and enforcing a Rs.10000 per-withdrawal limit and sufficient balance.

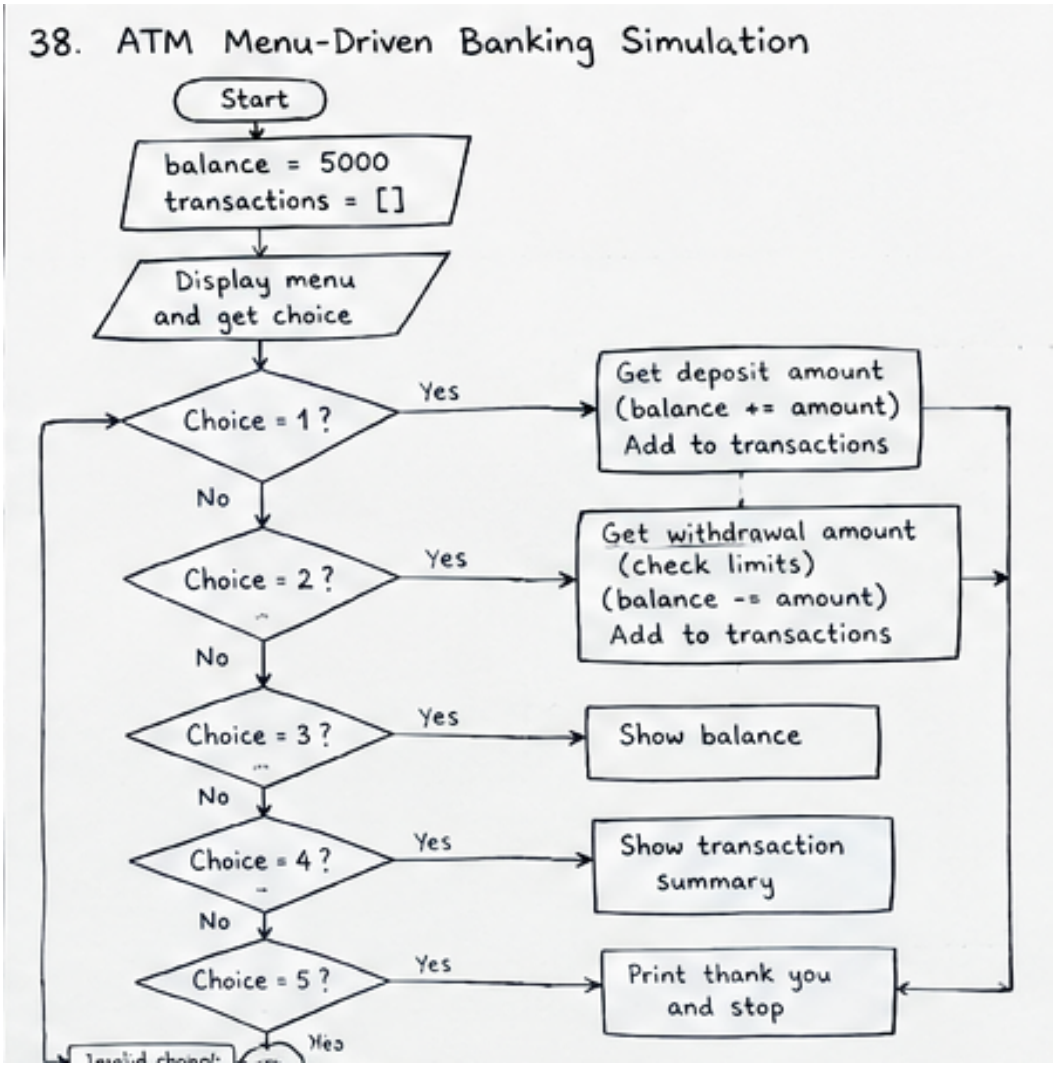
3. Assumptions

Opening balance is 5000; deposit must be > 0; withdrawal must be > 0, ≤ 10000 and ≤ balance; every successful transaction is recorded.

4. Algorithm

1. Initialise balance and empty transaction list
2. Loop: show menu and read choice
3. 1 Deposit: validate > 0, add to balance, record
4. 2 Withdraw: check > 0, ≤ 10000, ≤ balance, subtract, record
5. 3 Show balance
6. 4 Print all transactions and final balance
7. 5 Exit and stop loop; otherwise 'Invalid choice'

5. Flowchart



6. Pseudocode

```
balance <- 5000 ; transactions <- []
WHILE True
  SHOW menu ; READ choice
  IF choice=1: READ amt; IF amt>0 balance+=amt, log ELSE invalid
  ELIF choice=2: READ amt
     IF amt<=0 invalid; ELIF amt>10000 limit exceeded
     ELIF amt>balance insufficient; ELSE balance-=amt, log
  ELIF choice=3 PRINT balance
  ELIF choice=4 PRINT transactions, balance
  ELIF choice=5 PRINT thanks ; BREAK
  ELSE PRINT "Invalid choice"
```

7. Pvthon Source Code

```
balance = 5000.0   # opening balance (first lines of the file)
transactions = []

while True:
    print("\n1. Deposit")
    print("2. Withdraw")
    print("3. Check Balance")
    print("4. Transaction Summary")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        amount = float(input("Enter deposit amount: "))

        if amount > 0:
            balance += amount
            transactions.append("Deposit: " + str(amount))
            print("Deposit successful")
        else:
            print("Invalid amount")

    elif choice == "2":
        amount = float(input("Enter withdrawal amount: "))

        if amount <= 0:
            print("Invalid amount")
        elif amount > 10000:
            print("Withdrawal limit exceeded")
        elif amount > balance:
            print("Insufficient balance")
        else:
            balance -= amount
            transactions.append("Withdrawal: " + str(amount))
            print("Withdrawal successful")

    elif choice == "3":
        print("Balance:", balance)

    elif choice == "4":
        for item in transactions:
            print(item)
        print("Final balance:", balance)

    elif choice == "5":
        print("Thank you for using the ATM")
        break

    else:
        print("Invalid choice")
```

8. Test Cases

#	Input	Expected result
1	Deposit 2000, then Check Balance	Deposit successful; Balance: 7000.0
2	Withdraw 12000	Withdrawal limit exceeded
3	Withdraw 8000 with balance 7000	Insufficient balance
4	Withdraw -5 / Deposit 0	Invalid amount
5	Choice 9	Invalid choice
6	Choice 4 after the deposit	Deposit: 2000.0 / Final balance: 7000.0

9. Output

Code 38.py

Code

```python
transactions = []

while True:
    print("3. Check Balance")
    print("4. Transaction Summary")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        amount = float(input("Enter deposit amount: "))

        if amount > 0:
            balance += amount
            transactions.append("Deposit: " + str(amount))
            print("Deposit successful")
        else:
            print("Invalid amount")

    elif choice == "2":
        amount = float(input("Enter withdrawal amount: "))

        if amount <= 0:
            print("Invalid amount")
        elif amount > 10000:
            print("Withdrawal limit exceeded")
        elif amount > balance:
            print("Insufficient balance")
        else:
            balance -= amount
            transactions.append("Withdrawal: " + str(amount))
            print("Withdrawal successful")

    elif choice == "3":
        print("Balance:", balance)

    elif choice == "4":
        for item in transactions:
            print(item)
        print("Final balance:", balance)

    elif choice == "5":
        print("Thank you for using the ATM")
        break

    else:
        print("Invalid choice")
```

Output

```
/HP/OneDrive/Desktop/CSE-1/Dr.RVR NRI DTBU(Geethu)

1. Deposit
2. Withdraw
3. Check Balance
4. Transaction Summary
5. Exit
Enter your choice: 1
Enter deposit amount: 2000
Deposit successful

1. Deposit
2. Withdraw
3. Check Balance
4. Transaction Summary
5. Exit
Enter your choice: 3
Balance: 7000.0

1. Deposit
2. Withdraw
3. Check Balance
4. Transaction Summary
5. Exit
Enter your choice: 4
Deposit: 2000.0
Final balance: 7000.0

1. Deposit
2. Withdraw
3. Check Balance
4. Transaction Summary
5. Exit
Enter your choice: 5
Thank you for using the ATM
```

10. Conclusion

The menu-driven ATM handles all operations repeatedly with proper validation and keeps an accurate transaction history until the user exits.

39. Multi-Stage Examination Eligibility System

1. Problem Statement

Determine eligibility for a final examination using attendance, laboratory completion, internal assessment, and fee-clearance conditions, reporting every unmet requirement.

2. Objective

Check four conditions independently and list all that fail, instead of stopping at the first failure.

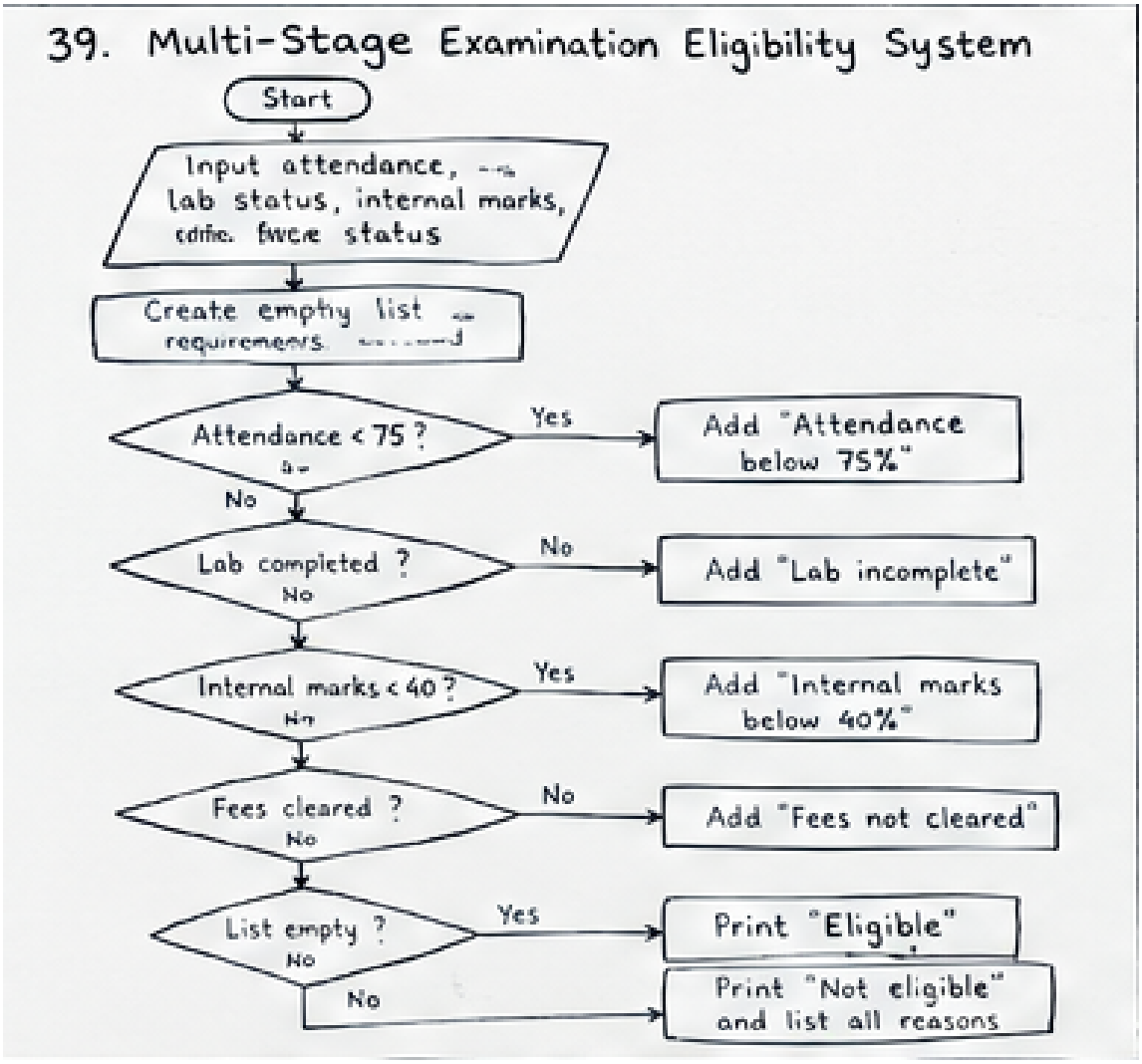
3. Assumptions

Attendance ≥ 75%, laboratory = yes, internal marks ≥ 40, fees cleared = yes; yes/no answers are case-insensitive.

4. Algorithm

1. Read attendance, lab, marks, fees
2. Create an empty requirements list
3. Append a message for each failed condition (independent ifs)
4. If list is empty print 'Eligible'
5. Else print 'Not eligible' and every message

5. Flowchart



6. Pseudocode

```
READ attendance, lab, marks, fees ; requirements <- []
IF attendance<75 ADD "Attendance is below 75%"
IF lab != yes ADD "Laboratory work is incomplete"
IF marks<40 ADD "Internal marks are below 40"
IF fees != yes ADD "Fees are not cleared"
IF requirements empty PRINT "Eligible"
ELSE PRINT "Not eligible" and each requirement
```

7. Pvthon Source Code

```
attendance = float(input("Enter attendance percentage: "))
lab = input("Laboratory completed (yes/no): ").lower()
marks = float(input("Enter internal marks: "))
fees = input("Fees cleared (yes/no): ").lower()

requirements = []

if attendance < 75:
    requirements.append("Attendance is below 75%")

if lab != "yes":
    requirements.append("Laboratory work is incomplete")

if marks < 40:
    requirements.append("Internal marks are below 40")

if fees != "yes":
    requirements.append("Fees are not cleared")

if len(requirements) == 0:
    print("Eligible for final examination")
else:
    print("Not eligible for final examination")
    for item in requirements:
        print(item)
```

8. Test Cases

#	Input	Expected result
1	65, no, 35, no	Not eligible; all 4 reasons listed
2	80, yes, 50, yes	Eligible for final examination
3	80, yes, 50, no	Not eligible; Fees are not cleared
4	75, yes, 40, yes	Eligible (boundary values pass)
5	74.9, YES, 45, yes	Not eligible; Attendance is below 75%

9. Output

Code 39.py

Code

```python
attendance = float(input("Enter attendance percentage: "))
lab = input("Laboratory completed (yes/no): ").lower()
marks = float(input("Enter internal marks: "))
fees = input("Fees cleared (yes/no): ").lower()

requirements = []

if attendance < 75:
    requirements.append("Attendance is below 75%")

if lab != "yes":
    requirements.append("Laboratory work is incomplete")

if marks < 40:
    requirements.append("Internal marks are below 40")

if fees != "yes":
    requirements.append("Fees are not cleared")

if len(requirements) == 0:
    print("Eligible for final examination")
else:
    print("Not eligible for final examination")
    for item in requirements:
        print(item)
```

Output

```
PS C:\Users\HP\OneDrive\Desktop\CSE-1\Dr.RVR NRI D
ive/Desktop/CSE-1/Dr.RVR NRI DTBU(Geethu)/39.py"
Enter attendance percentage: 65
Laboratory completed (yes/no): no
Enter internal marks: 35
Fees cleared (yes/no): no
Not eligible for final examination
Attendance is below 75%
Laboratory work is incomplete
Internal marks are below 40
Fees are not cleared
```

10. Conclusion

The system reports every unmet requirement at once, making it easy for a student to see exactly what must be fixed to become eligible.

40. Traffic Flow and Congestion Analysis

1. Problem Statement

Process vehicle counts across multiple roads and time intervals, calculate peak traffic periods, classify congestion levels, and identify roads exceeding capacity thresholds.

2. Objective

For 3 roads × 4 time slots find each road's peak period, classify every count and warn when capacity is exceeded.

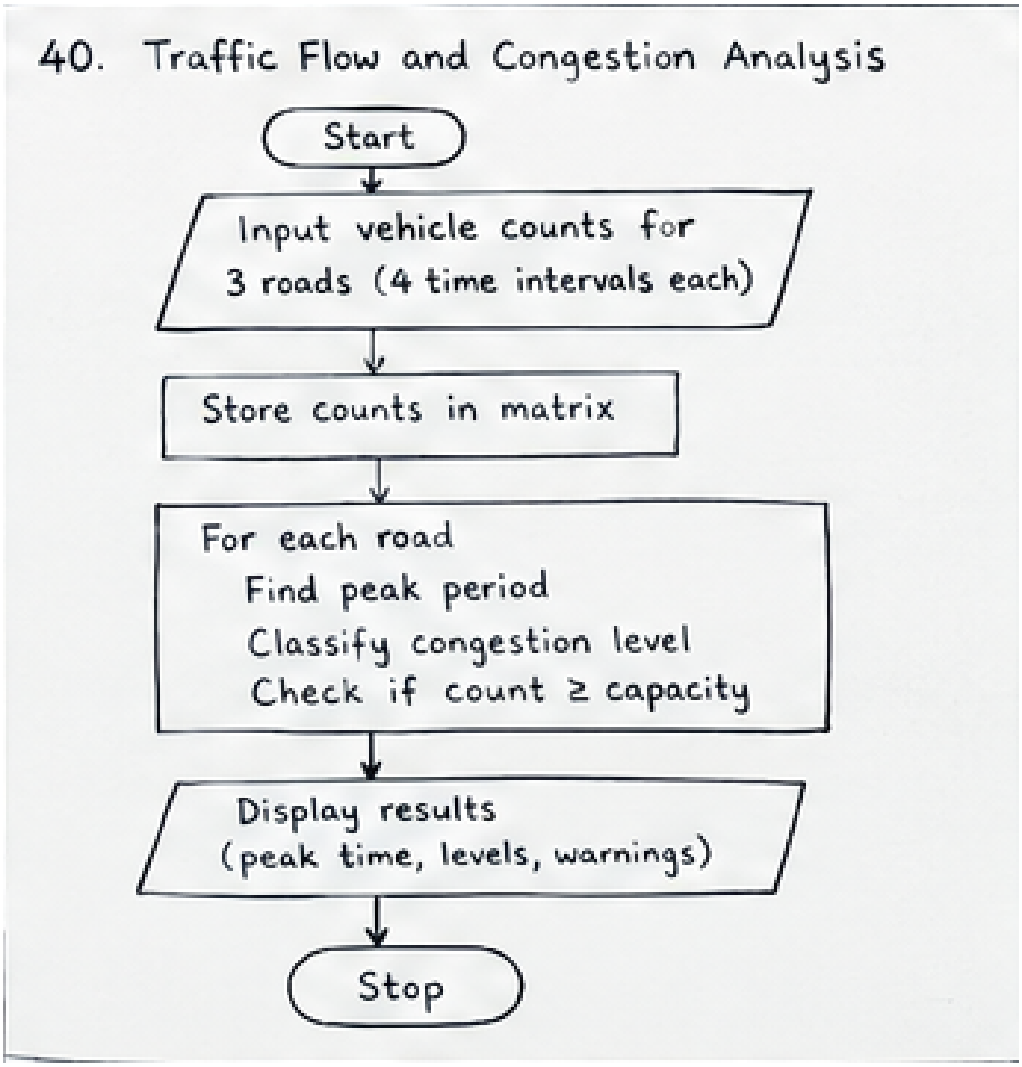
3. Assumptions

Counts are non-negative integers; Low < 30, Moderate < 60, High < 80, Critical ≥ 80; count ≥ 80 means the road exceeds capacity.

4. Algorithm

1. Store roads and time slots
2. Read counts for every road into a nested list
3. For each road find max count and its time slot (peak)
4. For each slot classify as Low/Moderate/High/Critical
5. Print a warning if count ≥ 80

5. Flowchart



6. Pseudocode

```
roads, times defined ; traffic <- []
FOR each road READ 4 counts, append row
FOR each road i
  peak <- MAX(traffic[i]) ; PRINT time of peak, peak
  FOR each slot j
    IF count<30 Low ELIF <60 Moderate ELIF <80 High ELSE Critical
    PRINT time, level
    IF count>=80 PRINT "Warning: Road exceeds capacity"
```

7. Pvthon Source Code

```
roads = ["Road 1", "Road 2", "Road 3"]
times = ["Morning", "Afternoon", "Evening", "Night"]
traffic = []

for road in roads:
    counts = []
    print("Enter vehicle counts for", road)

    for time in times:
        count = int(input(time + ": "))
        counts.append(count)

    traffic.append(counts)

for i in range(len(roads)):
    print("\n", roads[i])
    peak = max(traffic[i])
    print("Peak traffic:", times[traffic[i].index(peak)], peak)

    for j in range(len(times)):
        count = traffic[i][j]

        if count < 30:
            level = "Low"
        elif count < 60:
            level = "Moderate"
        elif count < 80:
            level = "High"
        else:
            level = "Critical"

        print(times[j], "-", level)

        if count >= 80:
            print("Warning: Road exceeds capacity")
```

8. Test Cases

#	Input	Expected result
1	Road 1: 120,140,170,180	Peak: Night 180; all Critical + warnings
2	Road 2: 120,140,130,120	Peak: Afternoon 140
3	Count 20 / 45	Low / Moderate
4	Count 79 / 80	High / Critical with warning

9. Output

Code 40.py

Code

```python
roads = ["Road 1", "Road 2", "Road 3"]
times = ["Morning", "Afternoon", "Evening", "Night"]
traffic = []

for road in roads:
    counts = []
    print("Enter vehicle counts for", road)

    for time in times:
        count = int(input(time + ": "))
        counts.append(count)

    traffic.append(counts)

for i in range(len(roads)):
    print("\n", roads[i])
    peak = max(traffic[i])
    print("Peak traffic:", times[traffic[i].index(peak)], peak)

    for j in range(len(times)):
        count = traffic[i][j]

        if count < 30:
            level = "Low"
        elif count < 60:
            level = "Moderate"
        elif count < 80:
            level = "High"
        else:
            level = "Critical"

        print(times[j], "-", level)

        if count >= 80:
            print("Warning: Road exceeds capacity")
```

Output

```
ive/Desktop/CSE-1/Dr.RVR NRI DTBU(Geethu)/
Enter vehicle counts for Road 1
Morning: 120
Afternoon: 140
Evening: 170
Night: 180
Enter vehicle counts for Road 2
Morning: 120
Afternoon: 140
Evening: 130
Night: 120
Enter vehicle counts for Road 3
Morning: 100
Afternoon: 110
Evening: 140
Night: 190

 Road 1
Peak traffic: Night 180
Morning - Critical
Warning: Road exceeds capacity
Afternoon - Critical
Warning: Road exceeds capacity
Evening - Critical
Warning: Road exceeds capacity
Night - Critical
Warning: Road exceeds capacity

 Road 2
Peak traffic: Afternoon 140
Morning - Critical
Warning: Road exceeds capacity
Afternoon - Critical
Warning: Road exceeds capacity
Evening - Critical
Warning: Road exceeds capacity
Night - Critical
Warning: Road exceeds capacity
```

10. Conclusion

The program identifies each road's peak period, classifies congestion correctly and flags roads that exceed capacity.

41. Nested List Classroom Occupancy Analysis

1. Problem Statement

Represent multiple classrooms and seating positions as nested lists; allocate students, preserve reserved seats, detect duplicate assignments, and calculate vacant capacity.

2. Objective

Store 2 classrooms × 3 seats, count vacant and reserved seats, detect duplicate student IDs and count occupied seats.

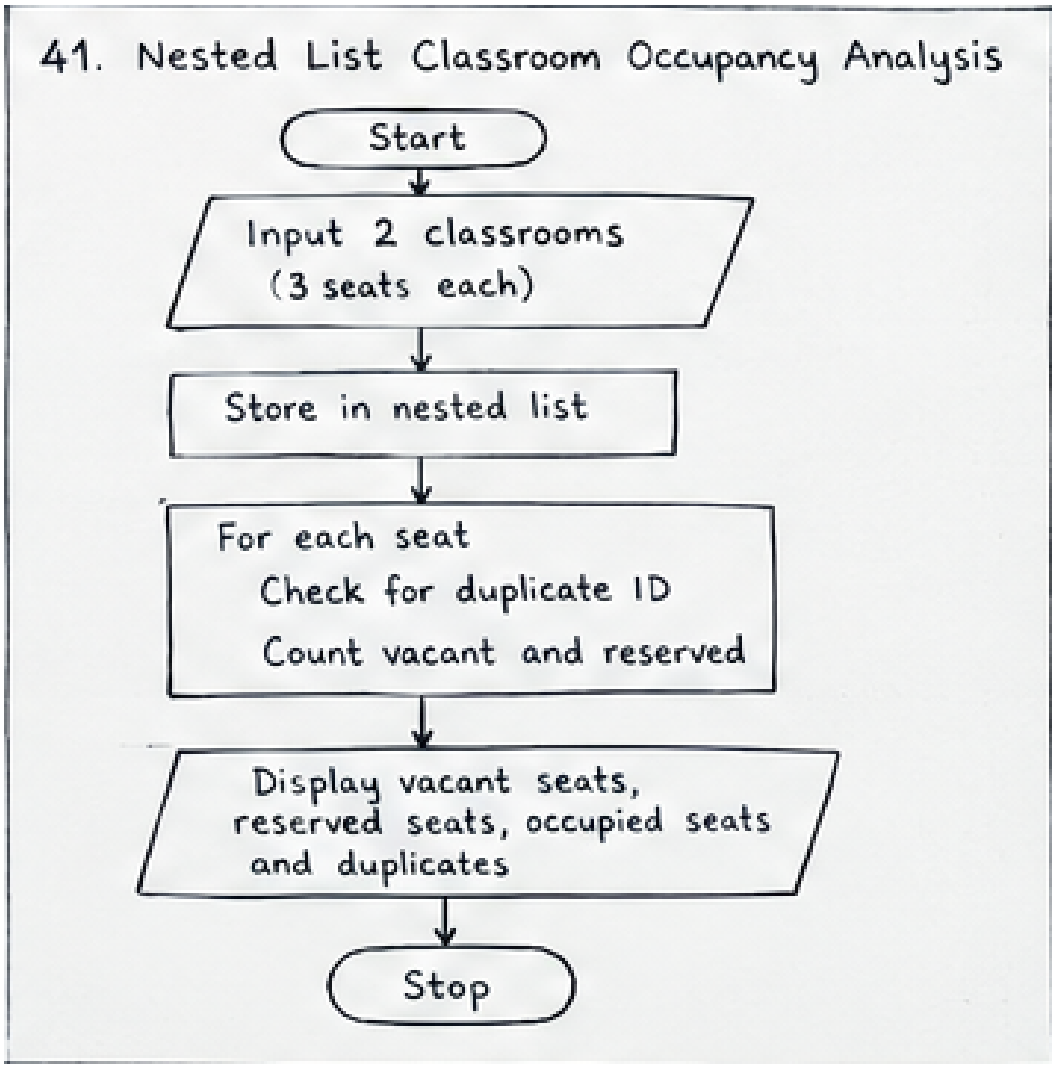
3. Assumptions

0 = vacant seat, –1 = reserved seat, any other number = student ID; a repeated ID is a duplicate and is not counted again.

4. Algorithm

1. Read seat values into a nested list
2. Loop over every seat
3. 0 → vacant++; –1 → reserved++
4. Otherwise if ID already in student_ids print duplicate, else store it
5. Print vacant, reserved and occupied (unique IDs)

5. Flowchart



6. Pseudocode

```
classrooms <- 2 rows of 3 seat values
vacant, reserved <- 0 ; student_ids <- []
FOR each seat
  IF seat=0 vacant++
  ELIF seat=-1 reserved++
  ELSE IF seat IN student_ids PRINT "Duplicate student ID", seat
       ELSE ADD seat to student_ids
PRINT vacant, reserved, LEN(student_ids)
```

7. Pvthon Source Code

```
classrooms = []

for i in range(2):
    seats = []
    print("Classroom", i + 1)

    for j in range(3):
        seat = int(input("Enter seat value: "))
        seats.append(seat)

    classrooms.append(seats)

student_ids = []
vacant = 0
reserved = 0

for i in range(len(classrooms)):
    for j in range(len(classrooms[i])):
        seat = classrooms[i][j]

        if seat == 0:
            vacant += 1
        elif seat == -1:
            reserved += 1
        else:
            if seat in student_ids:
                print("Duplicate student ID:", seat)
            else:
                student_ids.append(seat)

print("Vacant seats:", vacant)
print("Reserved seats:", reserved)
print("Occupied seats:", len(student_ids))
```

8. Test Cases

#	Input	Expected result
1	101,102,0 / -1,103,20	Vacant 1, Reserved 1, Occupied 4
2	101,101,0 / 5,6,7	Duplicate student ID: 101; Vacant 1, Reserved 0, Occupied 4
3	0,0,0 / 0,0,0	Vacant 6, Reserved 0, Occupied 0
4	-1,-1,-1 / -1,-1,-1	Reserved 6, Vacant 0, Occupied 0

9. Output

Code 41.py

Code

```python
classrooms = []

for i in range(2):
    seats = []
    print("Classroom", i + 1)

    for j in range(3):
        seat = int(input("Enter seat value: "))
        seats.append(seat)

    classrooms.append(seats)

student_ids = []
vacant = 0
reserved = 0

for i in range(len(classrooms)):
    for j in range(len(classrooms[i])):
        seat = classrooms[i][j]

        if seat == 0:
            vacant += 1
        elif seat == -1:
            reserved += 1
        else:
            if seat in student_ids:
                print("Duplicate student ID:", seat)
            else:
                student_ids.append(seat)

print("Vacant seats:", vacant)
print("Reserved seats:", reserved)
print("Occupied seats:", len(student_ids))
```

Output

```
PS C:\Users\HP\OneDrive\Desktop\CSE-1\Dr.
ive/Desktop/CSE-1/Dr.RVR NRI DTBU(Geethu)
Classroom 1
Enter seat value: 101
Enter seat value: 102
Enter seat value: 0
Classroom 2
Enter seat value: -1
Enter seat value: 103
Enter seat value: 20
Vacant seats: 1
Reserved seats: 1
Occupied seats: 4
```

10. Conclusion

Nested lists model the classrooms well; the program preserves reserved seats, flags duplicate IDs and reports accurate vacancy figures.

42. Student Marks Matrix Analysis

1. Problem Statement

Analyse a matrix of student marks across subjects, calculate row-wise and column-wise statistics, identify top performers, and detect subjects with low average performance.

2. Objective

Compute each student's total and average, each subject's average, flag subjects with average < 40 and find the top performer.

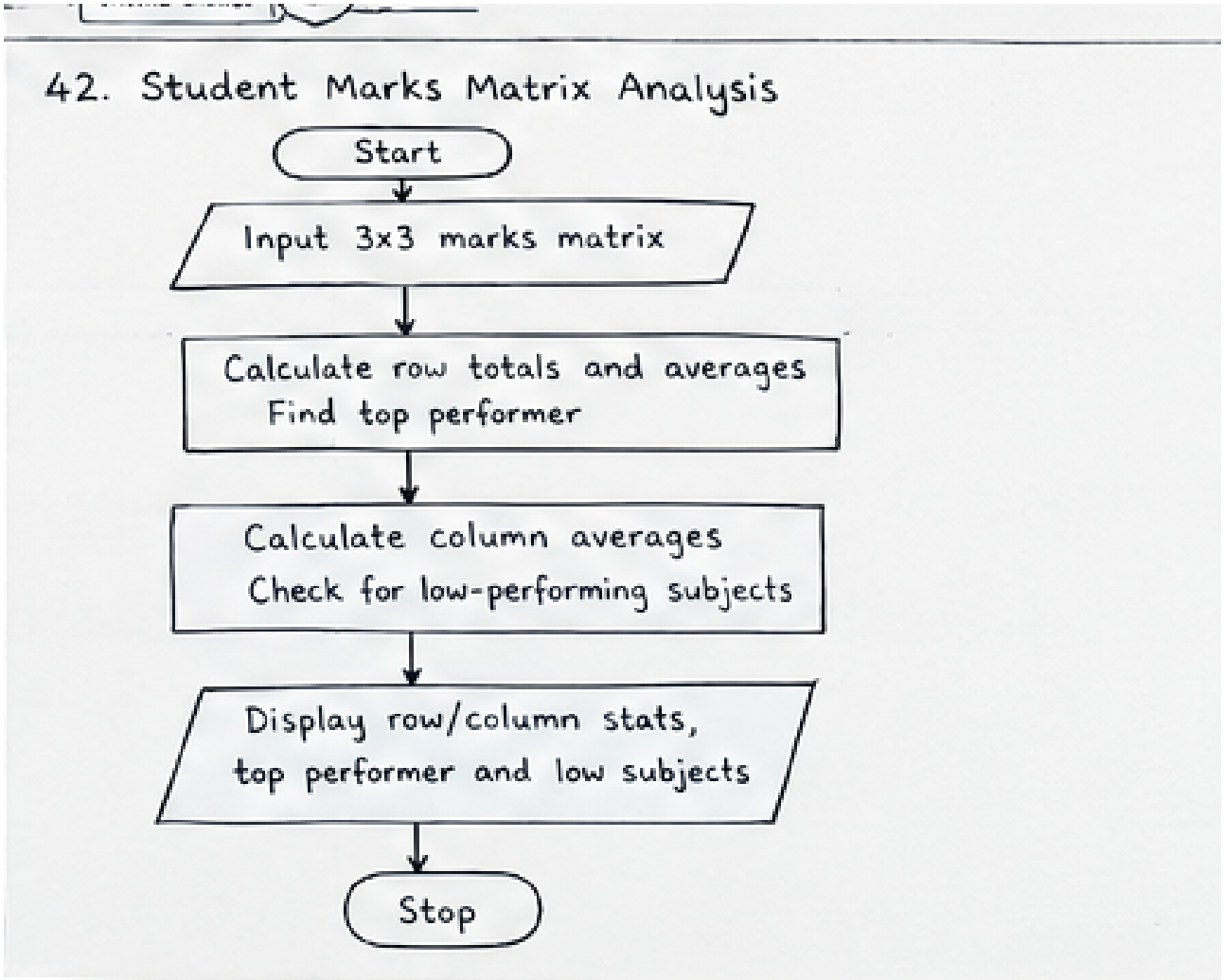
3. Assumptions

3 students × 3 subjects, integer marks; low-performing subject means average below 40; on a tie the first student is the top performer.

4. Algorithm

1. Read marks into a 3×3 nested list
2. Row-wise: total and average per student
3. Column-wise: average per subject; flag if < 40
4. Find the maximum total and its student number

5. Flowchart



6. Pseudocode

```
READ marks[3][3]
FOR each student i: total <- SUM(marks[i]); PRINT total, total/3
FOR each subject j
  total <- SUM of marks[i][j] over i ; average <- total/3
  PRINT average ; IF average<40 PRINT "Low-performing subject"
top <- INDEX(MAX(totals)) ; PRINT top+1
```

7. Pvthon Source Code

```
marks = []

for i in range(3):
    row = []
    print("Enter marks for student", i + 1)

    for j in range(3):
        value = int(input("Enter marks: "))
        row.append(value)

    marks.append(row)

totals = []

for i in range(3):
    total = sum(marks[i])
    totals.append(total)
    print("Student", i + 1, "total:", total)
    print("Average:", total / 3)

for j in range(3):
    total = 0

    for i in range(3):
        total += marks[i][j]

    average = total / 3
    print("Subject", j + 1, "average:", average)

    if average < 40:
        print("Low-performing subject")

top = totals.index(max(totals))
print("Top performer: Student", top + 1)
```

8. Test Cases

#	Input	Expected result
1	70,90,45 / 33,42,13 / 66,90,99	Totals 205, 88, 255; subject avgs 56.33, 74.0, 52.33; Top: Student 3
2	30,60,70 / 20,50,80 / 40,90,90	Subject 1 average 30.0 → Low-performing subject; Top: Student 3
3	All marks 50	All averages 50.0; Top performer: Student 1 (tie)

9. Output

Code 42.py

Code

```python
marks = []

for i in range(3):
    row = []
    print("Enter marks for student", i + 1)

    for j in range(3):
        value = int(input("Enter marks: "))
        row.append(value)

    marks.append(row)

totals = []

for i in range(3):
    total = sum(marks[i])
    totals.append(total)
    print("Student", i + 1, "total:", total)
    print("Average:", total / 3)

for j in range(3):
    total = 0

    for i in range(3):
        total += marks[i][j]

    average = total / 3
    print("Subject", j + 1, "average:", average)

    if average < 40:
        print("Low-performing subject")

top = totals.index(max(totals))
print("Top performer: Student", top + 1)
```

Output

```
PS C:\Users\HP\OneDrive\Desktop\CSE-1\Dr.RVR NRI
ive/Desktop/CSE-1/Dr.RVR NRI DTBU(Geethu)/42.py"
Enter marks for student 1
Enter marks: 70
Enter marks: 90
Enter marks: 45
Enter marks for student 2
Enter marks: 33
Enter marks: 42
Enter marks: 13
Enter marks for student 3
Enter marks: 66
Enter marks: 90
Enter marks: 99
Student 1 total: 205
Average: 68.33333333333333
Student 2 total: 88
Average: 29.333333333333332
Student 3 total: 255
Average: 85.0
Subject 1 average: 56.333333333333336
Subject 2 average: 74.0
Subject 3 average: 52.333333333333336
Top performer: Student 3
```

10. Conclusion

Row-wise and column-wise analysis gives complete student and subject statistics and correctly identifies the top performer and weak subjects.

43. Multi-Branch Sales Matrix Comparison

1. Problem Statement

Store product sales across branches and days in nested lists, calculate product totals, compare branches, and identify the best-performing product for each branch.

2. Objective

For 3 products × 2 branches × 3 days, compute product totals per branch, each branch's total and its best product.

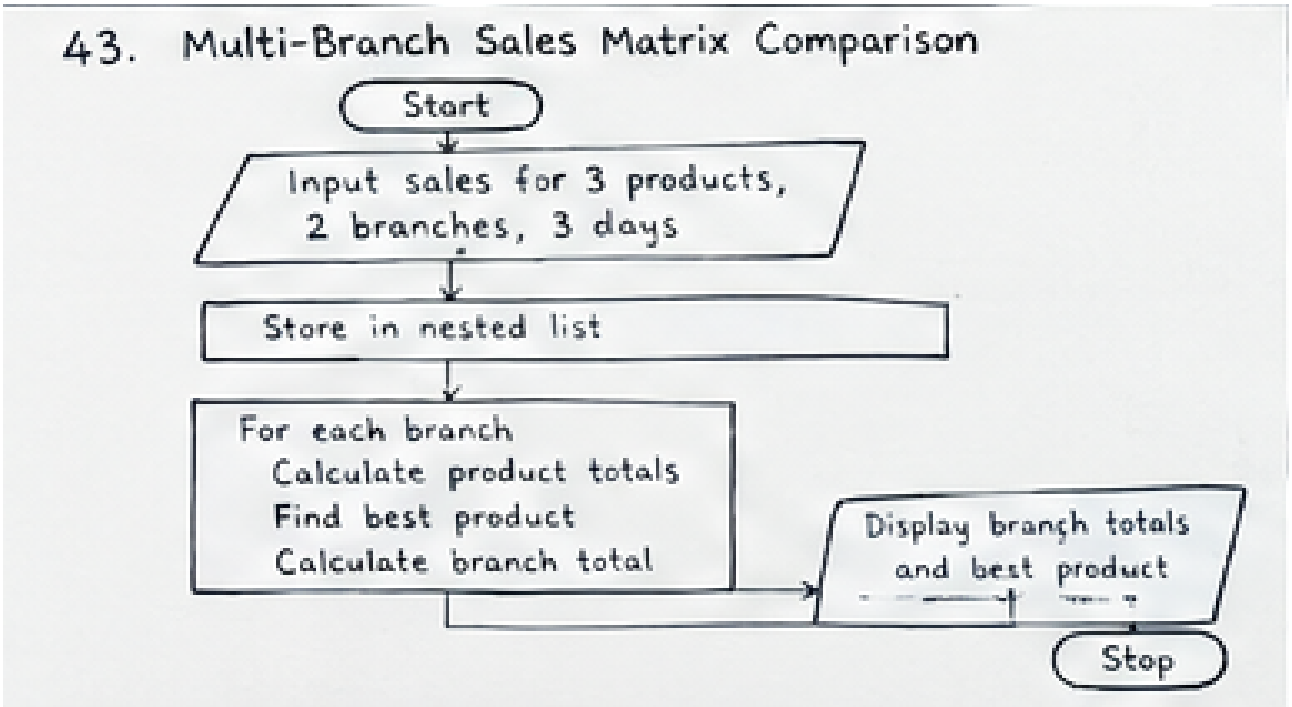
3. Assumptions

Daily sales are integers; the best product is the one with the highest 3-day total in that branch; ties go to the first product.

4. Algorithm

1. Read sales into a 3-level nested list [product][branch][day]
2. For each branch loop over products and sum the 3 days
3. Add to branch total; track highest product total
4. Print product totals, branch total and best product

5. Flowchart



6. Pseudocode

```
READ sales[product][branch][day]
FOR each branch j
  branch_total <- 0 ; best_sales <- -1
  FOR each product i
    total <- SUM(sales[i][j]) ; branch_total += total ; PRINT total
    IF total > best_sales best_sales <- total ; best_product <- i
  PRINT branch_total, products[best_product]
```

7. Pvthon Source Code

```
products = ["Product A", "Product B", "Product C"]
branches = ["Branch 1", "Branch 2"]
sales = []

for i in range(3):
    product_sales = []
    print("Enter sales for", products[i])

    for j in range(2):
        branch_sales = []
        print(branches[j])

        for k in range(3):
            amount = int(input("Enter daily sales: "))
            branch_sales.append(amount)

        product_sales.append(branch_sales)

    sales.append(product_sales)

for j in range(2):
    branch_total = 0
    best_product = 0
    best_sales = -1

    for i in range(3):
        total = sum(sales[i][j])
        branch_total += total

        print(products[i], branches[j], "total:", total)

        if total > best_sales:
            best_sales = total
            best_product = i

    print(branches[j], "total sales:", branch_total)
    print("Best product:", products[best_product])
```

8. Test Cases

#	Input	Expected result
1	Branch 1: A 120×3, B 100×3, C 100×3	A 360, B 300, C 300; total 960; Best: Product A
2	Branch 2: A 110,100,100; B 100,1001,100; C 1000,100,100	A 310, B 1201, C 1200; total 2711; Best: Product B
3	All sales 100	Every product 300; Best: Product A (tie, first wins)

9. Output

Code 43.py

Code

```python
products = ["Product A", "Product B", "Product C"]
branches = ["Branch 1", "Branch 2"]
sales = []

for i in range(3):
    product_sales = []
    print("Enter sales for", products[i])

    for j in range(2):
        branch_sales = []
        print(branches[j])

        for k in range(3):
            amount = int(input("Enter daily sales: "))
            branch_sales.append(amount)

        product_sales.append(branch_sales)

    sales.append(product_sales)

for j in range(2):
    branch_total = 0
    best_product = 0
    best_sales = -1

    for i in range(3):
        total = sum(sales[i][j])
        branch_total += total

        print(products[i], branches[j], "total:", total)

        if total > best_sales:
            best_sales = total
            best_product = i

    print(branches[j], "total sales:", branch_total)
    print("Best product:", products[best_product])
```

Output

```
Enter sales for Product A
Branch 1
Enter daily sales: 120
Enter daily sales: 120
Enter daily sales: 120
Branch 2
Enter daily sales: 110
Enter daily sales: 100
Enter daily sales: 100
Enter sales for Product B
Branch 1
Enter daily sales: 100
Enter daily sales: 100
Enter daily sales: 100
Branch 2
Enter daily sales: 100
Enter daily sales: 1001
Enter daily sales: 100
Enter sales for Product C
Branch 1
Enter daily sales: 100
Enter daily sales: 100
Enter daily sales: 100
Branch 2
Enter daily sales: 1000
Enter daily sales: 100
Enter daily sales: 100
Product A Branch 1 total: 360
Product B Branch 1 total: 300
Product C Branch 1 total: 300
Branch 1 total sales: 960
Best product: Product A
Product A Branch 2 total: 310
Product B Branch 2 total: 1201
Product C Branch 2 total: 1200
```

10. Conclusion

The program summarises sales per product and branch, compares branch totals and picks each branch's best-performing product.

44. Hospital Ward Occupancy Monitoring

1. Problem Statement

Analyse a nested list of bed occupancy counts by ward and day, calculate utilization percentages, detect capacity violations, and identify peak-demand periods.

2. Objective

For 3 wards × 3 days compute utilization %, flag days when occupied beds exceed capacity and find the peak demand ward and day.

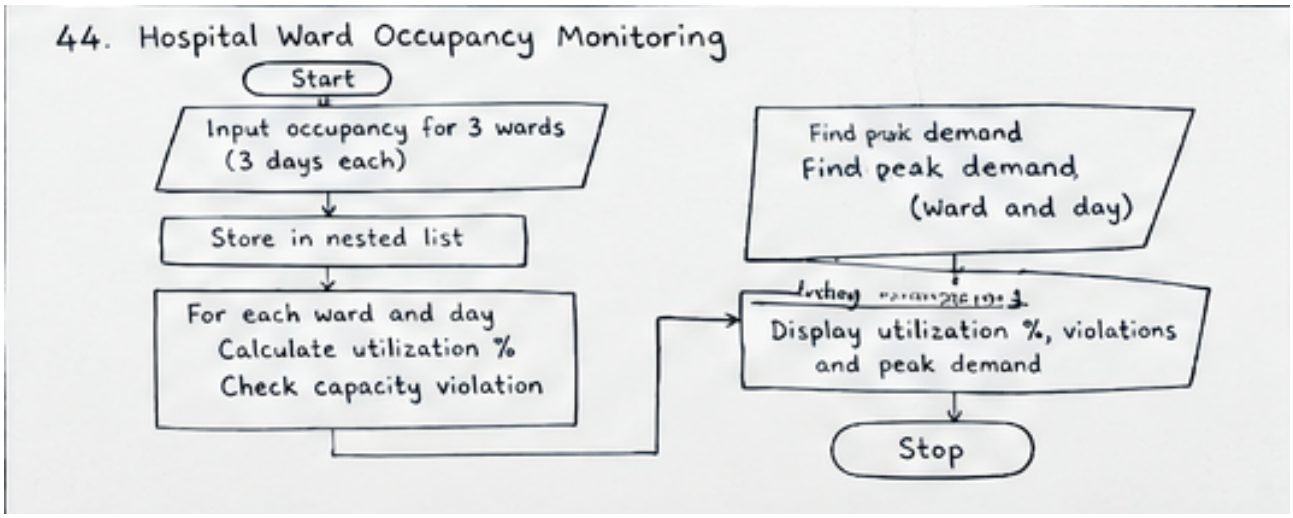
3. Assumptions

Ward capacities are 10, 15 and 20; utilization = occupied / capacity × 100; violation when occupied > capacity; peak is the highest bed count.

4. Algorithm

1. Define wards and capacity list
2. Read occupied beds into a 3×3 nested list
3. Compute utilization % for each ward/day and print
4. Print 'Capacity violation' if count > capacity
5. Scan for the maximum count and record ward and day

5. Flowchart



6. Pseudocode

```
wards, capacity defined ; READ occupancy[3][3]
FOR each ward i, day j
  pct <- occupancy[i][j]/capacity[i]*100 ; PRINT pct
  IF occupancy[i][j] > capacity[i] PRINT "Capacity violation"
peak <- -1
FOR each ward i, day j IF occupancy[i][j] > peak: update peak, ward, day
PRINT peak, ward, day
```

7. Pvthon Source Code

```
wards = ["Ward 1", "Ward 2", "Ward 3"]
capacity = [10, 15, 20]
occupancy = []

for i in range(3):
    row = []
    print("Enter occupancy for", wards[i])

    for j in range(3):
        count = int(input("Enter occupied beds: "))
        row.append(count)

    occupancy.append(row)

for i in range(3):
    print("\n", wards[i])

    for j in range(3):
        count = occupancy[i][j]
        percentage = count / capacity[i] * 100

        print("Day", j + 1, "utilization:", percentage, "%")

        if count > capacity[i]:
            print("Capacity violation")

peak = -1
peak_ward = ""
peak_day = 0

for i in range(3):
    for j in range(3):
        if occupancy[i][j] > peak:
            peak = occupancy[i][j]
            peak_ward = wards[i]
            peak_day = j + 1

print("Peak demand:", peak)
print("Ward:", peak_ward)
print("Day:", peak_day)
```

8. Test Cases

#	Input	Expected result
1	Ward 1: 10,12,13	100%, 120% + violation, 130% + violation
2	Ward 2: 14,12,10	93.33%, 80%, 66.67%; no violation
3	Ward 3: 10,10,11	50%, 50%, 55%
4	Overall (sample data)	Peak demand: 14, Ward 2, Day 1
5	Ward 3: 20,20,20	100% each, no violation (not above capacity)

9. Output

Code 44.py

Code

```python
wards = ["Ward 1", "Ward 2", "Ward 3"]
capacity = [10, 15, 20]
occupancy = []

for i in range(3):
    row = []
    print("Enter occupancy for", wards[i])

    for j in range(3):
        count = int(input("Enter occupied beds: "))
        row.append(count)

    occupancy.append(row)

for i in range(3):
    print("\n", wards[i])

    for j in range(3):
        count = occupancy[i][j]
        percentage = count / capacity[i] * 100

        print("Day", j + 1, "utilization:", percentage, "%")

        if count > capacity[i]:
            print("Capacity violation")

peak = -1
peak_ward = ""
peak_day = 0

for i in range(3):
    for j in range(3):
        if occupancy[i][j] > peak:
            peak = occupancy[i][j]
            peak_ward = wards[i]
            peak_day = j + 1

print("Peak demand:", peak)
print("Ward:", peak_ward)
print("Day:", peak_day)
```

Output

```
Enter occupancy for Ward 1
Enter occupied beds: 10
Enter occupied beds: 12
Enter occupied beds: 13
Enter occupancy for Ward 2
Enter occupied beds: 14
Enter occupied beds: 12
Enter occupied beds: 10
Enter occupancy for Ward 3
Enter occupied beds: 10
Enter occupied beds: 10
Enter occupied beds: 11

 Ward 1
Day 1 utilization: 100.0 %
Day 2 utilization: 120.0 %
Capacity violation
Day 3 utilization: 130.0 %
Capacity violation

 Ward 2
Day 1 utilization: 93.33333333333333 %
Day 2 utilization: 80.0 %
Day 3 utilization: 66.66666666666666 %

 Ward 3
Day 1 utilization: 50.0 %
Day 2 utilization: 50.0 %
Day 3 utilization: 55.00000000000001 %
Peak demand: 14
Ward: Ward 2
Day: 1
```

10. Conclusion

The monitoring program reports utilization, detects over-capacity days and locates the peak-demand ward and day for better resource planning.

45. Weekly Electricity Consumption Matrix

1. Problem Statement

Process appliance-wise electricity readings across seven days, calculate daily and appliance totals, and identify consumption exceeding specified limits.

2. Objective

For 3 appliances × 7 days compute weekly totals per appliance, daily totals, flag days above 100 units and find the peak day.

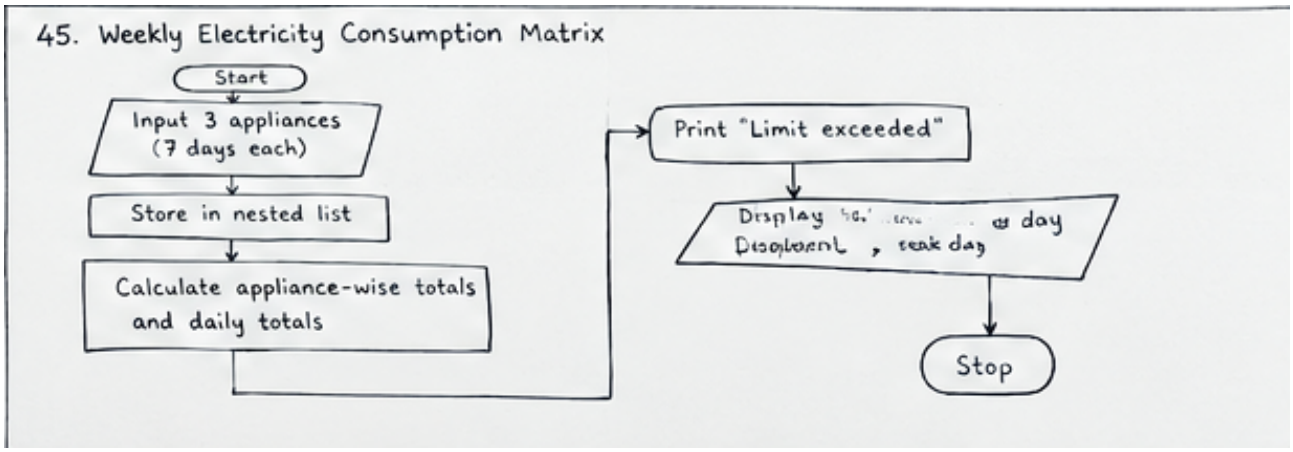
3. Assumptions

Readings are non-negative units (float); the daily limit is 100 units; on equal totals the earliest day is the peak day.

4. Algorithm

1 Read 7 daily readings for Fan, AC and Refrigerator
2 Print weekly total per appliance
3 For each day add the 3 appliances; print; warn if > 100
4 Track the highest daily total and its day number
5 Print highest consumption and peak day

5. Flowchart



6. Pseudocode

```
appliances defined ; READ consumption[3][7]
FOR each appliance i PRINT SUM(consumption[i])
FOR each day j
  daily_total <- SUM over i of consumption[i][j] ; PRINT
  IF daily_total>100 PRINT "Consumption limit exceeded"
peak <- -1
FOR each day j: total <- column sum ; IF total>peak update peak, peak_day
PRINT peak, peak_day
```

7. Pvthon Source Code

```
appliances = ["Fan", "AC", "Refrigerator"]
consumption = []

for i in range(3):
    row = []
    print("Enter weekly readings for", appliances[i])

    for j in range(7):
        value = float(input("Enter units consumed: "))
        row.append(value)

    consumption.append(row)

for i in range(3):
    print(appliances[i], "weekly total:", sum(consumption[i]))

for j in range(7):
    daily_total = 0

    for i in range(3):
        daily_total += consumption[i][j]

    print("Day", j + 1, "total:", daily_total)

    if daily_total > 100:
        print("Consumption limit exceeded")

peak = -1
peak_day = 0

for j in range(7):
    total = 0

    for i in range(3):
        total += consumption[i][j]

    if total > peak:
        peak = total
        peak_day = j + 1

print("Highest consumption:", peak)
print("Peak day:", peak_day)
```

8. Test Cases

#	Input	Expected result
1	Fan 3×7, AC 2×7, Fridge 2×7	Weekly 21.0, 14.0, 14.0; every day 7.0; Peak day 1
2	Day 3: Fan 50, AC 40, Fridge 20 (others small)	Day 3 total 110.0 → Consumption limit exceeded; Peak day 3
3	Day total exactly 100	No warning (limit is strictly > 100)
4	All readings 0	Totals 0.0; Peak day 1

9. Output

Code 45.py

Code

```python
appliances = ["Fan", "AC", "Refrigerator"]
consumption = []

for i in range(3):
    row = []
    print("Enter weekly readings for", appliances[i])

    for j in range(7):
        value = float(input("Enter units consumed: "))
        row.append(value)

    consumption.append(row)

for i in range(3):
    print(appliances[i], "weekly total:", sum(consumption[i]))

for j in range(7):
    daily_total = 0

    for i in range(3):
        daily_total += consumption[i][j]

    print("Day", j + 1, "total:", daily_total)

    if daily_total > 100:
        print("Consumption limit exceeded")

peak = -1
peak_day = 0

for j in range(7):
    total = 0

    for i in range(3):
        total += consumption[i][j]

    if total > peak:
        peak = total
        peak_day = j + 1

print("Highest consumption:", peak)
print("Peak day:", peak_day)
```

Output

```
Enter weekly readings for Fan
Enter units consumed: 3
Enter units consumed: 3
Enter units consumed: 3
Enter units consumed: 3
Enter units consumed: 3
Enter units consumed: 3
Enter units consumed: 3
Enter weekly readings for AC
Enter units consumed: 2
Enter units consumed: 2
Enter units consumed: 2
Enter units consumed: 2
Enter units consumed: 2
Enter units consumed: 2
Enter units consumed: 2
Enter weekly readings for Refrigerator
Enter units consumed: 2
Enter units consumed: 2
Enter units consumed: 2
Enter units consumed: 2
Enter units consumed: 2
Enter units consumed: 2
Enter units consumed: 2
Fan weekly total: 21.0
AC weekly total: 14.0
Refrigerator weekly total: 14.0
Day 1 total: 7.0
Day 2 total: 7.0
Day 3 total: 7.0
Day 4 total: 7.0
Day 5 total: 7.0
Day 6 total: 7.0
Day 7 total: 7.0
Highest consumption: 7.0
Peak day: 1
PS C:\Users\HP\OneDrive\Desktop\CSE-1\Dr.RVR NRI DTBU(Geethu)>
```

10. Conclusion

The program summarises appliance-wise and day-wise electricity use, warns when the daily limit is crossed and finds the peak day.